

WORKSHOP PARA ARQUITECTOS · PARTE 2

Mateu al barro

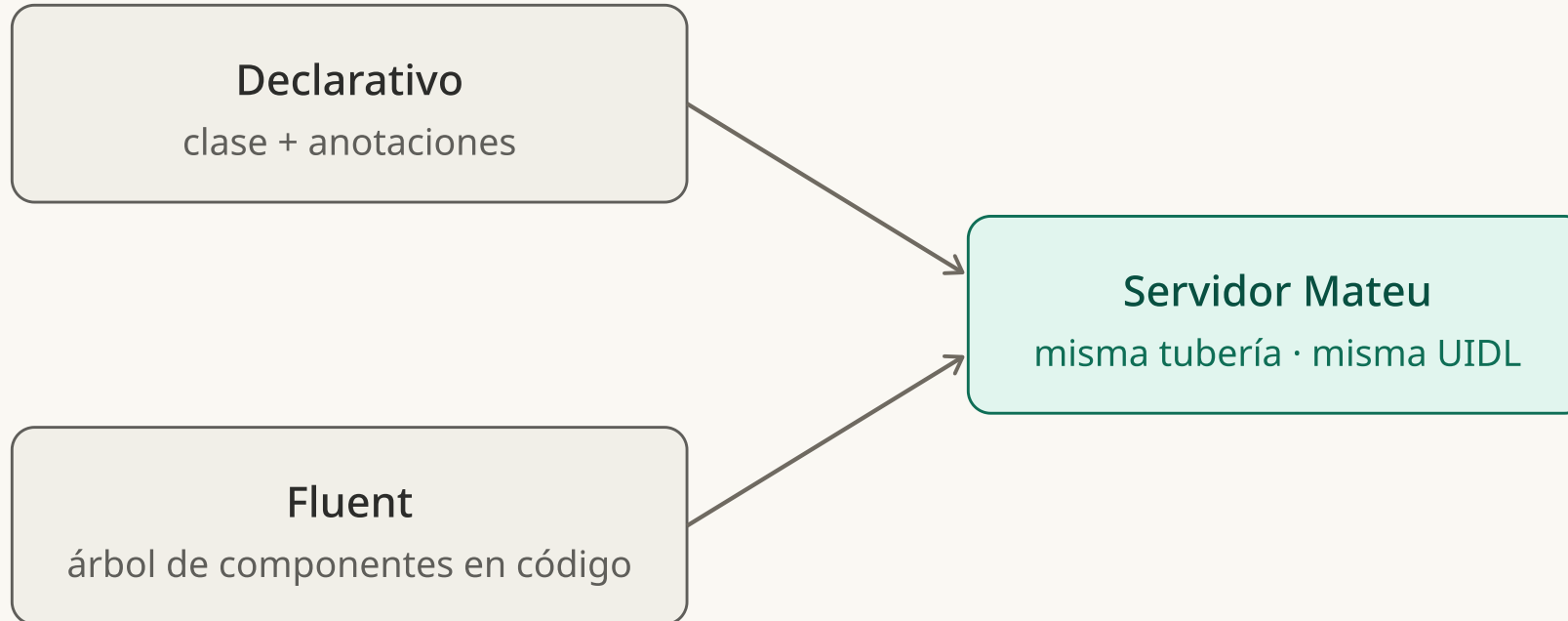
Del campo a la federación — esta vez, escribiendo el código

De la parte 1 a la parte 2

En la parte 1 vimos el **qué** y el **porqué**: la UI es data, el servidor es stateless, el renderer pinta la UIDL. Hoy bajamos al **cómo**.

- Definir la UI: **fluent** y **declarativo**
- Tres casos: **formulario**, **CRUD** y **formularios parciales**
- **Orquestadores** y **patrones de UX**: wizard, reglas, eventos, atajos
- **Vocabulario de dominio** y **extensión**: anotaciones, tipos, adaptadores, web components
- **Arquitectura**, despliegue, **federación** y multiplataforma
- Desarrollo con **IA**: estrategia y control

Dos formas de escribir la misma UI



Dos sintaxis, un solo motor. Eliges por **caso**, no por arquitectura: ambas acaban en la misma UIDL.

Declarativo para el 90% del backoffice; fluent cuando quieres controlar el árbol pieza a pieza.

Fluent: tú construyes el árbol

```
COMPONENTTREESUPPLIER · COMPONENT(HTTPREQUEST)
```

```
@UI("/counter")
```

```
public class Counter implements ComponentTreeSupplier {
```

```
    int count = 0;
```

```
    Counter increment() { count++; return this; }
```

```
    public Component component(HttpServletRequest req) {
```

```
        return new VerticalLayout(
```

```
            new Text("${state.count}"),
```

```
            new Button("Increment", () -> new
```

```
                State(increment()))
```

```
        );
```

```
    }
```

```
}
```

El estado es el campo (count); el binding es `${state.count}`; la acción es un **Callable** que devuelve el nuevo State. Sigue stateless.

Fluent: builders para todo

LISTING.BUILDER() · MISMA UI, CONTROL TOTAL

```
return Listing.builder()
    .title("Reservas")
    .searchable(true)
    .columns(List.of(
        GridColumn.builder().id("localizador").label("Loc.").sortable(true).build(),
        GridColumn.builder().id("hotel").label("Hotel").build()))
    .toolbar(List.of(Button.builder().label("Exportar").actionId("export").build()))
    .build();
```

- VerticalLayout · Card · Badge · Text · Chart · Avatar · Form · Listing · PageView...
- **Cuándo fluent:** componentes a medida, layouts dinámicos, contenido que depende de datos en runtime

Declarativo: la clase ES la pantalla

RECORDATORIO DE LA PARTE 1

```
@UI("")
@Title("My first Mateu app")
public class Home {
    @NotEmpty String name; // editable: hay un botón
    @Button Message greet() { return new Message("Hello " +
name); }
}
```

No describes widgets: describes **campos** (estado), **métodos** (acciones) y **anotaciones** (intención). Mateu deduce el resto.

Genera **exactamente la misma UIDL** que escribirías a mano con fluent — pero sin escribir el árbol.

Una clase → una pantalla

My first Mateu app

Name •

Mateu

Greet

Hello Mateu

Un campo con validación y un `@Button` → Mateu pinta el formulario; la acción devuelve un `Message` y sale como toast. Sin tocar HTML.

La anatomía de una pantalla declarativa

Estado

- › Los **campos** son las propiedades editables
- › El **tipo Java** infiere el control: `String`, `LocalDate`, `enum`, `boolean`...
- › `@Stereotype` fuerza el control cuando hace falta

Comportamiento

- › Los **métodos** son las acciones: `@Button`, `@Action`
- › Las **anotaciones** ajustan presentación y reglas
- › Validación con Bean Validation: `@NotEmpty`, `@Min`, `@Email`

Un solo sitio, una sola fuente de verdad. Sin DTOs de ida y vuelta, sin sincronizar front y back.

¿Fluent o declarativo?

Declarativo

- › Velocidad: el 90% del backoffice
- › Menos código, menos errores
- › Patrones ya resueltos (CRUD, wizard...)
- › Ideal para generar con IA

Fluent

- › Control total del árbol
- › Composiciones a medida
- › Contenido dinámico en runtime
- › Componentes propios / dashboards

Y se **mezclan**: una pantalla declarativa puede llevar un campo Component fluent (un gráfico, un avatar) embebido.

Caso 1 — Formulario: qué devuelve la acción

UN ROUND-TRIP, VARIOS EFECTOS

```
@Button Object guardar() {  
    repo.save(this);  
    return List.of(  
        new Message("Guardado"), // toast  
        new State(this), // re-hidrata la vista  
        URI.create("/clientes") // navega  
    );  
}
```

- **Message · State · URI** (o devolver un objeto = navegar a él) — o una `List` de varios
- La acción no manda píxeles: declara **efectos**; el renderer los aplica

Caso 2 — CRUD completo

```
AUTOCRUD<T> + CRUDREPOSITORY<T> + IDENTIFIABLE
```

```
@UI("/products")
```

```
public class Products extends AutoCrud<Products.Product> {  
    public CrudRepository<Product> repository() { return new  
        ProductRepository(); }
```

```
    record Product(  
        @NotEmpty @EditableOnlyWhenCreating String id,  
        @NotEmpty String name,  
        @Stereotype(FieldStereotype.textarea) @HiddenInList String  
description,  
        @Status(mappings = { @StatusMapping(from="OutOfStock", to=DANGER) })  
        ProductStatus status  
    ) implements Identifiable {}  
}
```

Una clase base + un repositorio = lista paginada, búsqueda, crear, editar, ver y borrar. Las anotaciones afinan cada columna y campo.

El otro lado: el repositorio

CRUDREPOSITORY<PRODUCT> — LA PERSISTENCIA LA PONES TÚ

```
static class ProductRepository implements CrudRepository<Product> {  
    static final Map<String,Product> db = new LinkedHashMap<>(...);  
  
    public Optional<Product> findById(String id) { return  
Optional.ofNullable(db.get(id)); }  
    public String save(Product e) { db.put(e.id(), e); return e.id(); }  
    public List<Product> findAll() { return db.values().stream().toList();  
}  
    public void deleteAllById(List<String> ids) { ids.forEach(db::remove);  
}  
}
```

Mateu no toca tu base de datos: le das un repositorio — JPA, Mongo, un API REST o un Map en memoria. La pantalla no cambia.

En vivo: /products — lista → crear → editar → borrar, todo generado.

...y esto es lo que se pinta

Products

[Do something on rows](#)[New](#)[Delete](#)

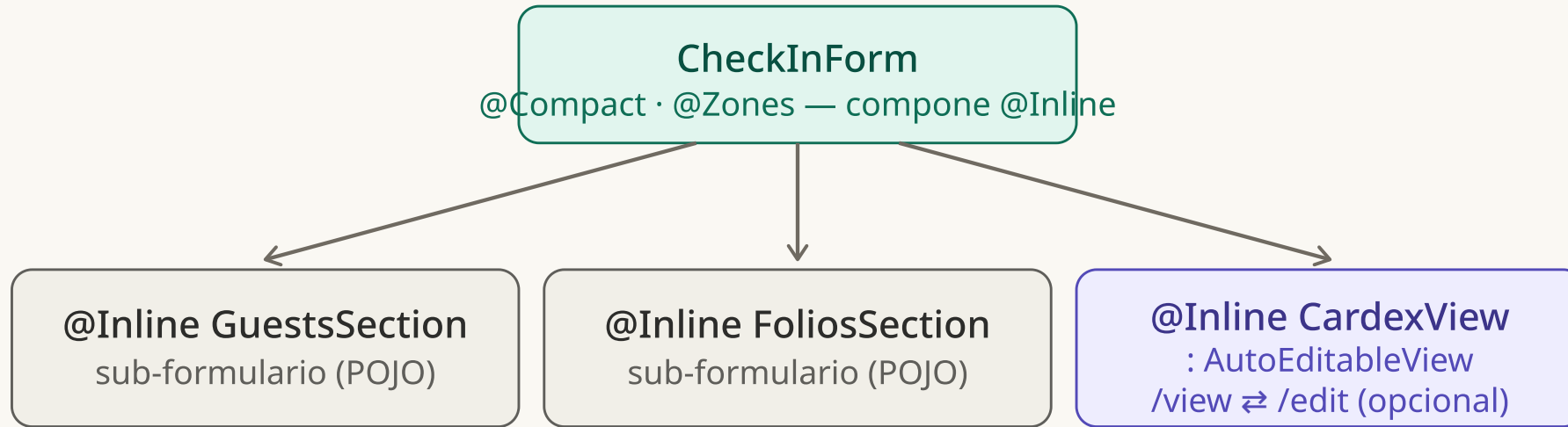
[Filters](#)[Search](#)

	Id	Name	Certified	Status	Action
☐	0130	Producto 130	✓	Available	...
☐	0010	Producto 10	✓	Available	...
☐	0131	Producto 131	—	Available	...
☐	0006	Producto 6	✓	Out of stock	...
☐	0127	Producto 127	—	Available	...
☐	0007	Producto 7	—	Available	...
☐	0128	Producto 128	✓	Available	...

[«](#)[<](#)Page 1 of 20[>](#)[»](#) | 200 items

La misma clase + repositorio → lista con búsqueda y filtros, crear/editar/borrar. Cero HTML, cero CSS.

Formularios

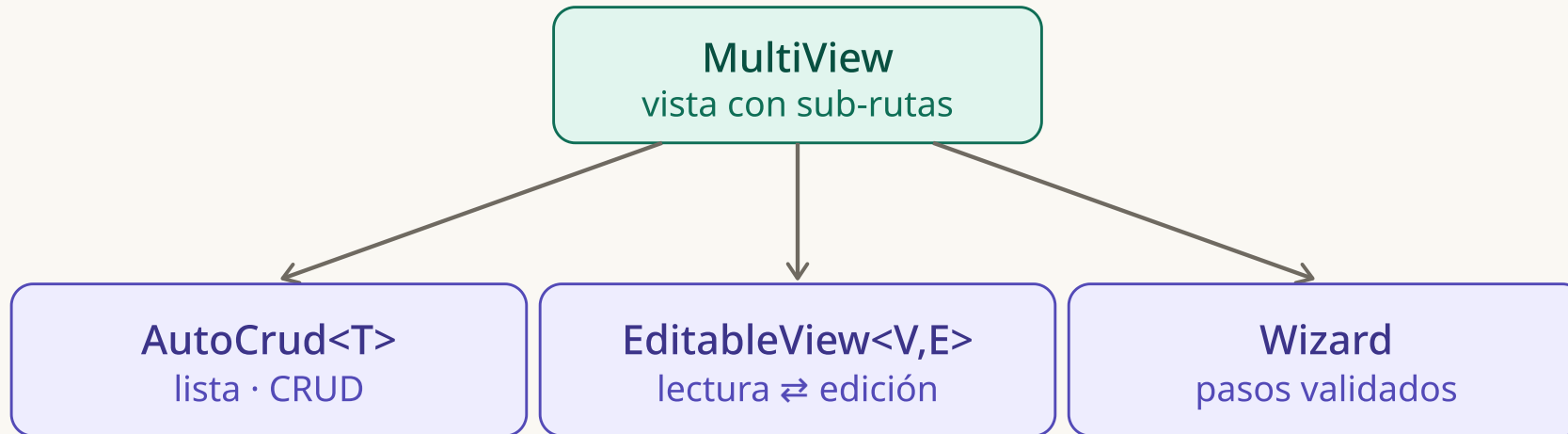


```
@UI("/checkin") @Compact @Zones(...)
public class CheckInForm {
    @Section("Huéspedes") @Inline GuestsSection huespedes; // POJO
    @Section("Folios") @Inline FoliosSection folios; // POJO
    @Section("Info cliente") @Inline CardexView cardex; // editable
}

// un sub-formulario, opcionalmente un orquestador editor:
class CardexView extends AutoEditableView<Cardex> {
    public Cardex load(HttpRequest r) { return repo.get(r); }
    public void persist(Cardex e, HttpRequest r) { repo.save(e); }
}
```

Una pantalla compone varios **sub formularios** con **@Inline** — cada uno una clase aparte. Los que necesitan

Orquestadores: patrones de UX en clases base



Los casos 2 y 3 —y el wizard que viene— son **lo mismo**: subclases de MultiView. Un orquestador es un **patrón de UX empaquetado**: extiendes la clase, rellenas los huecos (repositorio, pasos) y tienes el patrón entero.

Patrón: Wizard por pasos

EXTENDS WIZARD · CADA WIZARDSTEP ES UN PASO

```
@UI("/alta-socio")
public class AltaSocio extends Wizard {
    DatosPersonales datos;
    Direccion direccion;
    Resumen resumen;

    @WizardCompletionAction
    public Message completar() {
        resumen = new Resumen("Alta de " + datos.nombre());
        return new Message("Alta completada");
    }

    record DatosPersonales(@NotEmpty String nombre, @NotEmpty @Email String
email) implements WizardStep {}
    record Direccion(@NotEmpty String calle, String ciudad) implements
WizardStep {}
    @PlainText record Resumen(String texto) implements WizardStep {}
}
```

Los pasos son los campos, en orden de declaración. @WizardCompletionAction es el botón del penúltimo paso; el resultado es de solo lectura (@PlainText).

Wizard 1

Step 2

Age

—

+

Back

Next

o del wizard con barra de progreso y Back/Next — cada `WizardStep` es una pantalla; el avance inst siguiente.

Patrón: trabajo largo, async con Flux

LA ACCIÓN DEVUELVE UN FLUX → PROGRESO EN STREAMING

@Button

```
Flux<?> procesar() {  
    return LongTask.create("Procesando...")  
        .withProgressBar()  
        .done("Terminado", "Hecho")  
        .closeAfter(3)  
        .run(progress -> Flux.range(1, 10)  
            .delayElements(Duration.ofMillis(100))  
            .map(i -> progress.step("Procesados: " + i, i / 10d)));  
}
```

Devuelves un Flux y Mateu emite cada paso por SSE: la barra de progreso se actualiza sin bloquear. Sin WebSocket a mano, sin polling.

Variante simple: `return Flux....map(x -> new State(...))` empuja estados al formulario en vivo.

Patrón: triggers (eventos del ciclo de vida)

```
@TRIGGER — DISPARA UNA ACCIÓN SIN QUE EL USUARIO PULSE NADA

@UI("/pedidos")
@Trigger(type = TriggerType.OnLoad, actionId = "cargar")
public class Pedidos {
    List<PedidoRow> pedidos;

    Object cargar(HttpRequest req) {
        pedidos = servicio.ultimos();
        return new State(this);
    }
}
```

OnLoad · onSuccess · onError · OnValueChange · OnCustomEvent — enganchas lógica a momentos, no a clics.

Entre componentes: @Emit publican un evento y @SubscribeTo lo escucha — un panel se recarga cuando otro cambia.

Patrón: atajos de teclado

EN ACCIONES Y EN TABS

```
@Button @Action(shortcut = "ctrl+s")  
Message guardar() { return new Message("Guardado"); }  
  
@Tab(value = "Datos (alt+1)", shortcut = "alt+1") String nombre;  
@Tab(value = "Dirección (alt+2)", shortcut = "alt+2") String calle;
```

Modificadores ctrl · alt · shift · meta. Los atajos de tab se resuelven en el cliente, sin viaje al servidor.

Patrón: reglas (UI condicional)

```
@RULE — REACCIONA EN EL CLIENTE, SIN VIAJE AL SERVIDOR

@UI("/reserva")
@Rule(
    filter = "tipoCliente != 'EMPRESA'", // condición
    action = RuleAction.SetAttributeValue,
    fieldName = "cif", fieldAttribute = RuleFieldAttribute.hidden,
    value = "true", expression = "", actionId = "",
    result = RuleResult.Continue)
public class Reserva {
    String tipoCliente; // PARTICULAR | EMPRESA
    String cif; // se oculta salvo que sea empresa
}
```

Cuando filter se cumple, aplica una action sobre un atributo del campo: hidden · disabled · required · minLength · style · css... o RunAction/RunJS. Repetible con @Rules.

Patrón: comunicación entre componentes

@EMITS PUBLICA · @SUBSCRIBETO ESCUCHA — BUS DE EVENTOS EN EL CLIENTE

```
@Emits(name = "guests")
class GuestsSection {
    @OnRowSelected("onSel")
    List<Guest> guests;
    Object onSel(Guest g, HttpRequest r) {
        return UICommand.dispatchEvent("pax-selected", g.cardex());
    }
}

@SubscribeTo(event = "pax-selected", action = "reload", source =
COMPONENT, from = "guests")
class CardexView extends AutoEditableView<Cardex> { ... }
```

Seleccionas una fila en un panel y otro **se recarga solo**, sin navegar ni refrescar la página. Alcance del evento: DOCUMENT (global) · COMPONENT (filtra por el name del emisor) · SELF.

Modo @Compact: alta densidad

Check-in

LOCALIZADOR
RH-2026-83471

HOTEL
RIU05 Riu Palace Tenerife

AGENCIA
AG0042 Tui España

ESTADO
PENDING

ESTANCIA
2026-07-01 → 2026-07-04 · 3N

OCUPACIÓN
2 AD · 1 CH · 0 BB

RÉGIMEN
HB

TIPO COBRO
Pago en front

SALDO
0.00 EUR

Garantizada

Múltiple

VIP

Información general de la reserva

Tiempo esperando	Ref. tarifa	Tipo tarifa	Grupo res.	Grupo op.	Riu Class	Requiere
04:12	TF-99820-A	b2c2	—	—	Oro	—

Check-in

Nº habitación	Tipo hab. física	Tipo hab. contratada	Upgrade	Espera
—	DBL-V Doble vista mar	DBL Doble estándar	✓	✓
Deseos	Observaciones internas	Avisos		
Cama extra para niño. Habitación en planta alta si es posible.	Cumpleaños el 21/06.	Pendiente de validar documento del segundo adulto.		

Huéspedes

Confirmar check-in

No show

Lector documento

Tarjeta welcome

Deshacer check-in

Código WIFI

Apellidos	Nombre	Pax	Edad	Rég.	Nac.	Est.	Card.	Int.	Aviso	Obs. hotel
García Romero	Miguel	AD	41	HB	ES	R	✓	—	✓	Ciente VIP — bienvenida especial
García Romero	Laura	AD	39	HB	ES	P	✓	—	—	
García Soto	Daniela	CH	9	HB	ES	P	—	—	—	Cama extra solicitada

Información cliente

Editar cardex

Editar datos empresa

Info Cardex

Datos Empresa

Datos Tarjeta

Histórico cliente

Preferencias

Titular	Email
García Romero, Miguel	m.garcia@email.com
Teléfono / Fax	Dirección
+34 612 345 678	C/ Mayor 14, 3ºB · 28013 Madrid · ES España
Nac. / Idioma	F. nacimiento
ES Española / ES Español	1985-06-21 · MALE · Sevilla
Documento	Nº Riu Class
DNI · 12345678Z · cad. 2030-01-10	RC-7788991
Acepta publicidad	Acompañante
P	
Cardex provisional	

Importes

Tipo habitación	Precio estancia	Moneda	P/H
DBL Doble estándar	945,00	EUR	H
UPG Vista mar	120,00	EUR	H

Información habitación

Nº habitación	Dobles / Individuales	Estado	Checkout
—	0/2	SUCIA	✓
Observaciones	Averías		
Minibar repuesto. Caja fuerte operativa	Grifo del lavabo gotea — aviso a mantenimiento 18/06.		

Historial cliente

Tipo Riu Class	Último hotel	RPC	Repetido
ORO Riu Class Oro	RIU12	✓	14
Tipo cliente	Nº Attn H	Última habitación	
VIP	3	412	
Preferencias			
Habitación alta, vista mar, almohada viscoelástica. Cava de bienvenida.			

Folios / Anticipos

Límite crédito

Introducir cobro

Crédito cancelado	Imprimir recibo	Límite crédito	Tipo tarjeta
—	✓	1.500,00	VISA Crédito
4 últimos dígitos	Entrega a cuenta	Saldo pendiente	
4421	300,00	0,00	

Pantalla de check-in real con @Compact + @Zones: decenas de campos en dos columnas, cabecera con datos clave, badges e importes — todo en una vista sin scroll.

Vocabulario de dominio · anotaciones semánticas

En vez de repetir `@Lookup( . . . )` o `@Stereotype( . . . )` en cada pantalla, nombras el concepto **una vez** y lo reutilizas.

DEFINES LA ANOTACIÓN DE DOMINIO UNA VEZ...

```
@Lookup(search = ProveedorOptions.class, label = ProveedorLabel.class)
@Target(FIELD) @Retention(RUNTIME)
public @interface ProveedorId {}
```

...Y LA USAS COMO UN TIPO SEMÁNTICO

```
@ProveedorId String proveedorId;
@Importe BigDecimal total; // = @Stereotype(money)
@AccionGuardar Object guardar() {...} // = @Toolbar @Label("Guardar")
```

Funciona en **campo**, **método** y **clase** (`@PantallaCompacta = @Compact`). Como `@ProductId`, `@CustomerId`... construyes el lenguaje de tu dominio.

Excepción: las de routing (`@UI`, `@Route`) no son componibles — se resuelven en compilación.

Vocabulario de dominio · tipos con render propio

La otra vía: un **tipo** de negocio (una interfaz) y un **adapter** que dice cómo se pinta. Mateu ya trae alguno de fábrica — Amount (importe + divisa) — y así defines los tuyos.

TU INTERFAZ DE DOMINIO

```
public interface Coordenada { double lat(); double lng(); }
```

UN ADAPTER POR LA INTERFAZ

@Service

```
class CoordenadaAdapter implements ComponentAdapter<Coordenada> {  
    public Class<Coordenada> type() { return Coordenada.class; }  
    public AdaptedView adapt(Coordenada c, HttpRequest req) { /* mini-mapa  
lat/lng */ }  
    public Coordenada deserialize(Map<String,Object> s, HttpRequest req) {  
/* reconstruye */ }  
}
```

Casa por **asignabilidad**: un ComponentAdapter<Coordenada> pinta a Sede, Almacén, Cliente... cualquier campo cuyo tipo implemente Coordenada.

Extensión — Adaptadores custom

¿Y un objeto de dominio que **no** quieres anotar con Mateu? Le enseñas a Mateu a pintarlo, fuera de la clase.

```
COMPONENTADAPTER<T> · UN @SERVICE Y YA
```

```
@Service
```

```
public class PedidoAdapter implements ComponentAdapter<Pedido> {  
    public Class<Pedido> type() { return Pedido.class; }  
    public AdaptedView adapt(Pedido m, HttpRequest req) { /* componentes +  
estado */ }  
    public Pedido deserialize(Map<String, Object> state, HttpRequest req) {  
/* reconstruye */ }  
}
```

Mateu indexa todos los ComponentAdapter por su type(). El dominio queda **limpio**, sin acoplarse al framework de UI.

Extensión — Web components y CSS

Tu web component

- › El servidor emite un `ClientSideComponentDto`
- › Su `metadata` apunta a tu **custom element**
- › El renderer lo monta y le pasa estado/eventos
- › Para piezas que Mateu no tiene de fábrica

CSS a medida

- › `@Style("...")` — CSS inline / variables
- › `@CssClasses("...")` — clases propias
- › `@Link(rel="stylesheet", href="...")` — CSS externo (repetible con `@Links`)
- › `@Compact` — alta densidad (backoffice)
- › `StyleConstants` — presets reutilizables

Tres niveles de escape: **CSS** para el aspecto, **web component** para una pieza, **renderer** para todo el look & feel.

Seguridad declarativa

PROTEGER VISTAS Y OCULTAR POR ROL — SIN CÓDIGO DE SEGURIDAD EN EL FRONT

```
@UI("")
@KeycloakSecured(url = "https://kc/...", realm = "acme", clientId =
"backoffice")
public class Consola {
    @Menu Reservas reservas;
    @Menu @EyesOnly(roles = "admin") Auditoria auditoria; // solo admins
}

// también a nivel de campo o método:
@EyesOnly(roles = "finanzas") @Stereotype(FieldStereotype.money)
BigDecimal margen;
```

@EyesOnly acepta roles · groups · scopes · permissions, en campo, método o clase. No hay bugs de seguridad en el front... porque no hay front duplicado.

Lo que ya viene resuelto

- **Export Excel y PDF** en listados — de fábrica
- **i18n** — implementa `Translator` (o el `DefaultTranslator`)
- **@AutoSave**(debounceMillis, action) — guardado en caliente, sin botón
- **@App**(themeToggle=true) — claro/oscuro · variantes de menú (top · left · hamburguesa · tabs)
- **@Banner** · **@KPI** · **@BadgeInHeader** · **@Fab** — primitivas de cabecera y estado
- **@ConfirmOnNavigationIfDirty** — aviso de cambios sin guardar · **@UploadableImage** — subida sin endpoint

Todo declarativo, todo opt-in: lo enciendes con una anotación, no lo programas.

Dónde encaja en tu arquitectura

El ViewModel no es tu dominio

- › La clase `@UI` orquesta la UI
- › Detrás: use cases / servicios y el puerto `CrudRepository`
- › Encaja con hexagonal / DDD — el dominio queda limpio

Testable sin navegador

- › Las pantallas son **POJOs stateless**
- › Llamas al método y compruebas el **efecto** (`Message/State`)
- › Repos in-memory → tests rápidos y deterministas

El coste del stateless —rehidratar en cada request— es el precio de no tener sesiones de UI en el servidor y poder escalar en horizontal.

Cómo se sirve el renderer

El renderer es **estático** y agnóstico del backend; el API siempre habla `/mateu/v3/sync`. Tres formas de desplegarlo:

Todo-en-uno

- › Añades una dependencia: `io.mateu:vaadin-lit`
- › Spring Boot sirve el renderer desde el classpath — sin config
- › `baseUrl=""` → mismo origen, cero CORS
- › Un solo despliegue: renderer + API juntos

Split: CDN + API

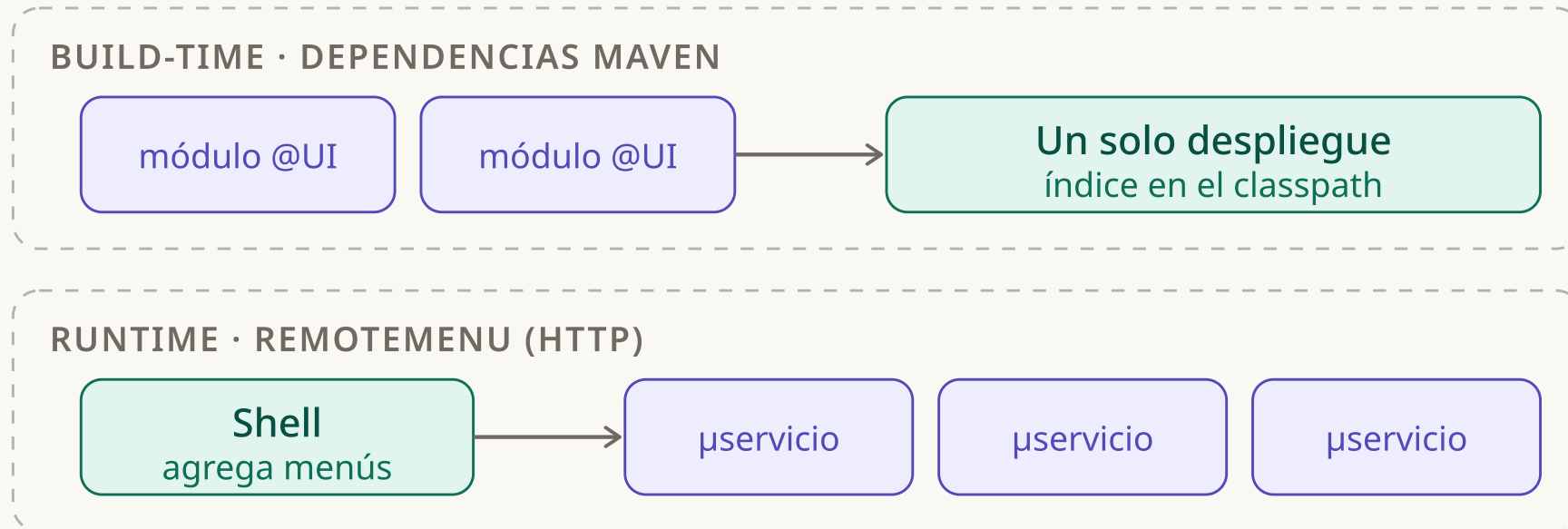
- › Publicas los assets estáticos en un **CDN** (cache en el edge)
- › `<mateu-ui baseUrl="https://api.tuapp.com">`
- › El API va a **tus servidores** Java
- › Escalas estático y backend por separado

Escritorio autocontenido

- › Renderer **nativo** (JavaFX/Compose) → **sin navegador**, app de escritorio real
- › O *web shell*: servidor en `localhost` que abre el navegador al arrancar
- › Un widget/acción lo cierra (p. ej. `System.exit`)
- › Herramienta local — nada que desplegar

Mismo renderer, mismo modelo de UI: eliges el acoplamiento según tu infra. Pasar de una a otra es, casi siempre, cambiar un atributo (`baseUrl`) — no tocas código de pantalla.

Federación: dos formas de componer



Mismo modelo de UI; dos acoplamientos: **compile-time** (un deploy cohesivo) o **runtime** (microservicios de verdad, despliegue independiente).

Build-time: composición por classpath

- Tu módulo añade `io.mateu:uidl` y escribe clases `@UI`
- El **annotation processor indexer** escribe `META-INF/mateu/ui-registrations` en el JAR
- La app depende de esos módulos y su AP **lee el índice del classpath**
- Genera los controladores del framework (MVC, WebFlux, Quarkus...) sin tener las fuentes

Empaquetas varios dominios @UI en un único despliegue. Ideal cuando los módulos viven en el mismo equipo.

Runtime: el shell agrega por HTTP

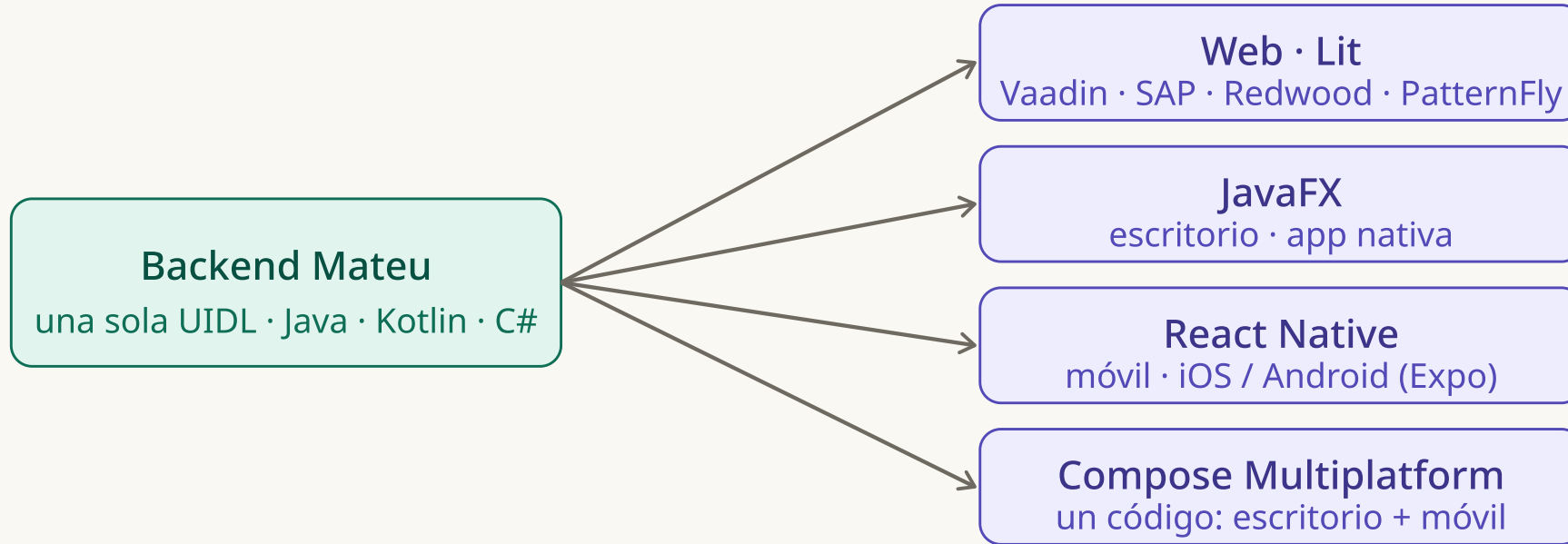
EL SHELL SÓLO DECLARA DE QUIÉN ES CADA PANTALLA

```
@UI("")
@Title("Consola")
public class Shell {
    @Menu RemoteMenu reservas = new
RemoteMenu("http://reservas:8081", "/reservas");
    @Menu RemoteMenu clientes = new
RemoteMenu("http://clientes:8082", "/clientes");
}
```

- El shell es **thin**: no hace de proxy, dice quién sirve cada UI
- Un microservicio publica una pantalla nueva y **aparece sin redesplegar el shell**

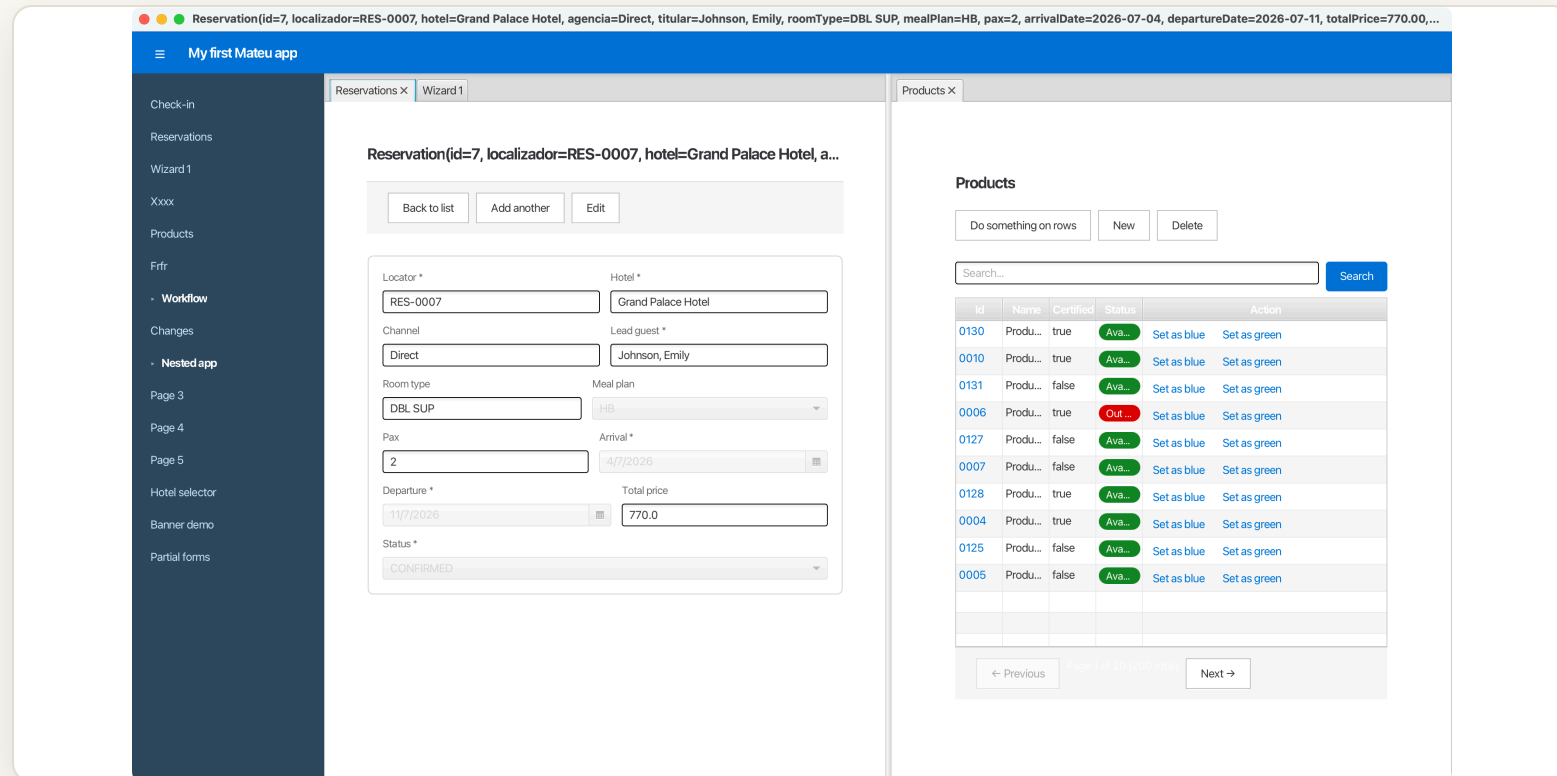
Esto cierra el círculo con la parte 1: el shell agrega UIs igual que el agente agrega MCPs.

Más allá del navegador



La UIDL es el contrato; el renderer es intercambiable y **no está atado a una plataforma** — Compose cubre escritorio y móvil con un solo código, y una plataforma puede servirse desde varias tecnologías. Cualquier cliente que hable el mismo API pinta tu app, **sin tocar el backend**.

Renderers nativos, de verdad



El mismo backend → app de **escritorio JavaFX** con pestañas y paneles acoplables. Y además: **React Native** (móvil) y **Compose Multiplatform** (Kotlin: un código → escritorio + iOS + Android).

Elige tu backend

Frameworks (JVM)

- › Spring **MVC** y **WebFlux**
- › **Quarkus**
- › **Micronaut**

Lenguajes

- › **Java**
- › **Kotlin** (JVM)
- › **C#** — Mateu.NET (ASP.NET) **EN CONSTRUCCIÓN**

Todos emiten el mismo `/mateu/v3/sync` → cualquier renderer pinta cualquier backend. La UIDL es el contrato, no el lenguaje.

¿Mateu o IA? La pregunta equivocada

No es «Mateu o IA». La IA no compite con Mateu — se **reubica**. Lo que cambia no es *si* usas IA, sino **el tamaño y el riesgo de lo que emite**.

IA genera React

- › El artefacto es el **código final**: JSX, hooks, estado, validaciones
- › Eso es lo que revisas, versionas, mantienes y **regeneras** a cada cambio
- › **Doble fuente de verdad**: dominio + UI, a resincronizar para siempre
- › Se aprueba «por fe»: nadie lee 2.000 líneas generadas

IA genera el metamodelo

- › La IA emite la **intención**: una clase anotada (~30 líneas)
- › Un **renderer determinista** aguas abajo la materializa
- › **Una sola verdad**: la UI se deriva del dominio; un campo nuevo se propaga solo
- › Revisable de verdad; la consistencia es del sistema, no de acertar 500 veces

El eje: **IA para intent** → **spec**, **generador determinista para spec** → **UI**. Contraintuitivo: cuanto **mejor** es la IA con React, **más fuerte** el caso de Mateu — producir código tiende a coste cero, pero *confiar* en él (revisar, mantener, que no derive) no baja. La IA resuelve lo barato; Mateu ataca lo caro.

Sin autoengaño: si tu valor es UX diferenciada y la UI es el producto, deja que la IA genere React. Mateu gana en CRUD/formularios/listas/master-detail, y cuando priorizas consistencia y longevidad sobre expresividad máxima.

Las reglas por encima: coordinar las UIs de la empresa

Construir *una* pantalla —con IA o sin ella— es el problema pequeño. El grande es que **todas** las UIs de la empresa converjan. Ahí Mateu no es un framework de pantallas: es la **capa que fija las reglas**.

- **Un contrato (UIDL) + un renderer** → N equipos, cientos de pantallas se comportan igual: la consistencia es **propiedad del sistema**, no de acertar pantalla a pantalla
- **Vocabulario de dominio compartido** → las convenciones de la empresa se codifican **una vez** (anotaciones semánticas) y todos las heredan
- **Puertos/SPIs estándar** (`CrudRepository` , `ListingBackend` , `Translator` , seguridad por roles, i18n) → cada elemento se enchufa igual; lo transversal, uniforme
- **Federación** (Maven + `RemoteMenu` / `MicroFrontend`) → muchos servicios/equipos componen **una** app coherente, y el design system se gobierna en un sitio

Por eso el debate no es «IA o no»: la IA **genera piezas**; Mateu **gobierna cómo encajan** — el contrato, el vocabulario y la composición que hacen converger las UIs de toda la empresa.

Desarrollo con IA: por qué Mateu encaja

- **Menos tokens** — la referencia entera son ~2,5k–9k tokens, no el universo HTML/CSS/JS/React
- **Más determinista** — dado el modelo Java, la UI está determinada; nada que "inventar" de estilos
- **Menos código no controlado** — anotaciones + tipos; el LLM no escribe miles de líneas de front
- **Más control del proyecto** — la salida compila o no; los errores son sintácticos, no "se ve raro"

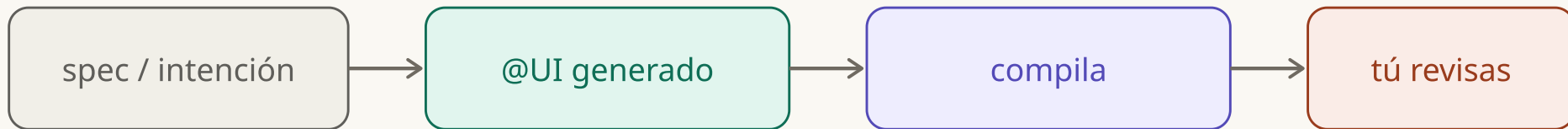
La UI es una **proyección** del modelo: justo lo que un LLM hace bien — transformar, no diseñar.

Desarrollo con IA: cómo se monta hoy

- **Referencia en contexto** — `mateu-ai-compact.md` (~2,5k) y `mateu-ai-full.md` (~9k)
- **Descubrimiento** — `llms.txt` según el estándar, para que el agente la encuentre
- **Integración** — Claude Projects, `.cursorrules`, `copilot-instructions.md`
- **Pides en lenguaje natural** — "CRUD de Product con id, name, price, active"

Ya empaquetado como **skills** de Claude Code: `mateu` (escribir pantallas), `mateu-scaffold` (cablear el build), `mateu-run` (ejecutar + captura), `mateu-federation` — flujos guiados y deterministas que se cargan bajo demanda, no solo un texto de referencia.

La estrategia: determinismo sobre código libre



El LLM produce poca superficie y muy acotada; tú mantienes el control del proyecto. Menos magia, más revisable.

ROADMAP

Lo que viene

Producto · antes de fin de 2026

- › Completar **todos los renderers** — SAP, Redwood, PatternFly y SLDS al nivel de Vaadin, y cerrar los nativos
- › Terminar el **backend C#** (Mateu.NET)

Ecosistema · adopción

- › Hablar con **Spring, Quarkus y Micronaut**
- › Mateu en el **Spring Initializr**
- › Mateu en la **documentación oficial** de esos frameworks

El objetivo: que arrancar con Mateu sea marcar una casilla en `start.spring.io`.

EN UNA FRASE

Declaras la intención. Mateu hace el resto.

- Fluent o declarativo, una clase es una pantalla — y ambas dan la misma UIDL.
- Formulario, CRUD y editor: patrones de UX empaquetados en clases abstractas.
- Extiendes con adaptadores, web components y CSS; compones por Maven o por HTTP.
- Y la IA encaja porque hay **poco código, determinista y bajo tu control**.

Cuándo sí: backoffice, herramientas internas, consolas. **Cuándo no:** cuando la UI es el producto.